TIVIDY Implementation Plan

How We Planned the Build

TIVIDY was built by seven people in dependency-ordered phases so that nobody waited on anyone. This document is the plan we worked from: who owned which pattern and file, which branch to use, and when a test could be pushed. Letters A to G are the team members, in the order listed in each section heading.

The goal is that nobody is ever blocked, nobody is overloaded, every pattern has one owner, and PRs roll out successfully. It is very important that you review PRs promptly and regularly, but it is even more important that you do indeed check the code, rather than just pressing approve.

An important reminder is that whilst implementation is occurring, diagrams are still a requirement for the practical.

Word	Plain meaning
Phase	A wave of work. Phase N only uses files finished in phases below N. Everyone in a phase can work at the same time (and may need to collaborate).
Integration branch	A shared branch for files that call each other in a circle. They only compile together, so they merge together.
Lead	The one person who opens the single PR from an integration branch into `main` once all helpers are merged in.
Collaborator	The other person (max 1) who also writes `.cpp` code for the same header. Talk before you start; agree who writes which functions.
CI	The automatic build and test run on every PR. A PR cannot merge if it is red.

Why only phase 1 can start today: `.cpp` files that call each other in a circle (e.g. `WorkflowInstance` and `FlowController`) cannot be built alone, so they travel together. Calling through a base class (e.g. a `WorkItem*`) needs no link to the child, which is why most files are free of each other.

1. The rules

- **Max 2 people per header.** The pair splits the functions by the file list in section 4 (each `.cpp` has one author).
- **One `.cpp` = one author.** Overloaded headers were given new `.cpp` names so two people never edit the same file.
- **Repo layout:** headers live in `include/`, `.cpp` files in `src/` (same sub-path), tests in `tests/`. `make test` builds every `tests/**/*.cpp` as its own GoogleTest binary linked to `libtividy.a`.
- **Branch names must match the repo check** `^(feature|fix|docs|refactor|test|chore)/[A-Za-z0-9._-]+$`. Code: `feature/p{phase}-{letter}-{slug}`. Tests: `test/p{phase}-{letter}-{slug}`. Integration: `feature/p{phase}-integration-{name}`.

- **PR titles** use `type: what changed` (`feat: add FlowController reactions`, `test: add State pattern tests`). Commits use the same form.
- **Every PR into `main` needs 4 approvals** (CONTRIBUTING.md) and green CI (Docker build + `make test`). Review each other's PRs promptly, because with 7 people each PR waits on 4 of the other 6. Reviewing is how you stay unblocked.
- **Code PRs go into `main`, except integration pieces,** which go into their `feature/p{phase}-integration-...` branch (no CI or approvals on those PRs: the workflows only run for PRs into `main`). Only the lead opens the PR from that branch into `main`.
- **A test PR may open only when every file it needs is already merged to `main`** (the `needs:` list in section 5). `make test` runs ALL tests, so a red test anywhere blocks everyone.
- **Every `.cpp` must compile on its own** (`make` builds all of `src/` into the library), even if it does not link yet. Only tests that reference a missing symbol fail to link.
- **Engine neutrality check:** `grep -riE "banking|dispute|chargeback|telecom|msisdn" include/engine src/engine include/ui src/ui` must print nothing.
- **Commit often** (each member needs 10+ commits and 3+ closed PRs in total; this plan gives everyone more than that).
- **Stuck for over 30 minutes?** Post in the group chat. Everyone's Phase N work is independent, so someone can always help.

2. Pattern owners (also write the `test_<pattern>.cpp`)

Person	Patterns owned	Their pattern tests
A	State, Iterator	`test_pattern_state.cpp`, `test_pattern_iterator.cpp`
B	Template Method, Chain of Responsibility	`test_pattern_chain_of_responsibility.cpp`, `test_pattern_template_method.cpp`
C	Composite, Prototype, Strategy	`test_pattern_strategy.cpp`, `test_pattern_composite.cpp`, `test_pattern_prototype.cpp`
D	Command, Observer, Visitor	`test_pattern_observer.cpp`, `test_pattern_visitor.cpp`, `test_pattern_command.cpp`
E	Builder, Abstract Factory, Memento	`test_pattern_memento.cpp`, `test_pattern_builder.cpp`, `test_pattern_abstract_factory.cpp`
F	Adapter, Facade, Decorator	`test_pattern_adapter.cpp`, `test_pattern_decorator.cpp`, `test_pattern_facade.cpp`
G	none (owns the industry packs and the TUI: they *use* every pattern)	smoke tests for banking, telecom, demo (shared with whoever is listed in section 5)

The pattern owner is *responsible* for the pattern being clearly visible in code and tested. They write most of that pattern's classes, but where a pattern spans a big file the work is shared (max 2 people).

Given that the team has agreed to make use of test-driven development, after implementing a feature or class, ensure to unit test it as far as possible in the relevant test_folder.cpp file. Each implementation should ideally come with an update to the testing file it references.

The distinguishing factor between regular testing and pattern testing is that pattern testing focuses on checking that specific functionality is met, whilst regular testing focuses on edge cases and ensuring there are no errors in the code.

For the pattern you are responsible for, you may implement your pattern tests against the functionality requirements in the project report at any time, but you may only push these when you have all the dependencies locally and the tests pass locally. The pattern tests do not depend on implementation. Since there is cross collaboration, once all the files you need for your pattern appear to be done, your tests should pass. If they do not, it is your responsibility to debug the pattern specific issues, or the functionality not met. Hence, even though we collaborate across the team to meet our deliverables, your pattern and its correctness remains your responsibility.

To allow more efficient coordination, you will notice that you will sometimes be asked to create a file that does not strictly match a header file. A good example of how this works is PR #9 (feat: validate workflow definitions), where a file WorkflowDefinitionValidation.cpp was created to implement parts of the WorkflowDefinition class. This helps with breaking up large files to distribute workload and it also helps with lifting circular dependencies within your own patterns (and for others).

3. Balance

Person	Code lines	Test lines	Total	Phase 1	2	3	4	5	6	7
A	427	310	737	62	190	245	100	80	60	0
B	535	230	765	60	210	205	190	50	50	0
C	420	290	710	65	155	170	170	90	60	0
D	485	260	745	60	235	110	110	90	140	0
E	420	320	740	95	185	140	170	0	90	60
F	480	240	720	35	160	95	180	0	250	0
G	640	60	700	320	0	190	130	0	60	0

Lines are estimates. The split also weighs difficulty (hard files such as `FlowController` count about 1.5 times), so totals are within about 5% by effort. G has nothing in phases 2 and 5 on purpose: those phases wait for the engine, and other work was completed during Phase 0, allowing Phase 1 to begin. In this time, G will be doing management and assistance across the team. Phase 7 is the TUI folder test only; then everyone helps with the report, Doxygen and demo rehearsal. There will be no more implementation beyond Phase 6, only debugging as needed.

You do not necessarily need to be idle between phases or at any point in the project. You can help other team members that are lagging behind, and work independently where you can. We aimed to finish through Phase 6 quickly, at roughly two to three phases per day, to leave room for error.

4. Your phase-by-phase tasks

A. Ishmael

Phase 1

Branch	File(s)	Lines	With	Into
`feature/p1-a-command-base`	`src/engine/commands/Command.cpp`	12	-	`main`
`feature/p1-a-director-base`	`src/engine/definition/WorkflowDirector.cpp`	10	E	`main`
`feature/p1-a-item-memento`	`src/engine/items/WorkItemMemento.cpp`	10	-	`main`
`feature/p1-a-work-item-base`	`src/engine/items/WorkItem.cpp`	30	-	`main`

- `Command.cpp` : CommandResult::ok/failure and Command dtor.
- `WorkflowDirector.cpp` : NEW FILE. WorkflowDirector dtor + build() template method.
- `WorkItemMemento.cpp` : Private ctor (status Created, counters 0, Accepted outcome, empty ids) and dtor.
- `WorkItem.cpp` : WorkItem dtor + defaults: children(), add() (Unsupported), createMemento() (empty), restore(), isReversible() (true), assignedAt()/dueAt().

Phase 2

Branch	File(s)	Lines	With	Into
`feature/p2-a-action-history`	`src/engine/commands/ActionHistory.cpp`	45	-	`feature/p2-integration-runtime`
`feature/p2-a-deadline-monitor`	`src/engine/runtime/DeadlineMonitor.cpp`	35	-	`feature/p2-integration-runtime`
`feature/p2-a-stage-item`	`src/engine/items/StageItem.cpp`	110	-	`main`

- `ActionHistory.cpp` : Ctor/dtor, execute (needs Running), undoLast, size, describeAll.
- `DeadlineMonitor.cpp` : Ctor and sweep() (report each lapse once).
- `StageItem.cpp` : Derived status() (6 rules), add (duplicate id check), find, cancel/skip cascades, throwing ops, clone, accept, createIterator, bind.

Phase 3

Branch	File(s)	Lines	With	Into
`feature/p3-a-decorator-secure`	`src/engine/items/WorkItemDecorator.cpp`	95	B	`main`
`test/p3-a-state`	`tests/test_pattern_state.cpp`	90	-	`main`
`test/p3-a-iterator`	`tests/test_pattern_iterator.cpp`	60	-	`main`

- `WorkItemDecorator.cpp` : WorkItemDecorator forwarders + SecureWorkItem.

Phase 4

Branch	File(s)	Lines	With	Into
`test/p4-a-items`	`tests/test_engine_items.cpp`	50	-	`main`
`test/p4-a-definition`	`tests/test_engine_definition.cpp`	50	-	`main`

Phase 5

Branch	File(s)	Lines	With	Into
`feature/p5-a-tividy-actions`	`src/engine/facade/TividyActions.cpp`	80	C	`feature/p5-integration-tividy`

- `TividyActions.cpp` : NEW FILE. perform, execute, undo, commandFactory, workFor, history, progress, signal, advanceTime, now, participants, connectors.

Phase 6

Branch	File(s)	Lines	With	Into
`test/p6-a-demo-pack-smoke`	`tests/test_industry_demo.cpp`	60	-	`main`

Phase 7: nothing new. Help review PRs and write docs, etc.

B. Aidan

Phase 1

Branch	File(s)	Lines	With	Into
`test/p1-b-chain-of-responsibility`	`tests/test_pattern_chain_of_responsibility.cpp`	60	-	`main`

Phase 2

Branch	File(s)	Lines	With	Into
`feature/p2-b-concrete-states`	`src/engine/items/WorkItemStates.cpp`	150	E	`feature/p2-integration-state-machine`
`feature/p2-b-state-helpers`	`src/engine/items/WorkItemStateHelpers.cpp`	60	E	`feature/p2-integration-state-machine`

- `WorkItemStates.cpp` : NEW FILE. The 9 concrete states exactly as in the transition table (incl. expire).
- `WorkItemStateHelpers.cpp` : NEW FILE. moveTo, emit, setAssignee, runWork, recordResult, bump counters, requireAssignee.

Phase 3

Branch	File(s)	Lines	With	Into
`feature/p3-b-command-decorators`	`src/engine/commands/CommandDecorators.cpp`	50	C	`main`
`feature/p3-b-decorator-validated`	`src/engine/items/WorkItemValidation.cpp`	55	A	`main`
`feature/p3-b-workflow-library`	`src/engine/definition/WorkflowLibrary.cpp`	30	-	`main`
`test/p3-b-template-method`	`tests/test_pattern_template_method.cpp`	70	-	`main`

- `CommandDecorators.cpp` : NEW FILE. CommandDecorator (forwarding) + RetryingCommand.
- `WorkItemValidation.cpp` : NEW FILE. ValidationRule dtor, RequiredDataRule, ValidatedWorkItem.
- `WorkflowLibrary.cpp` : add (validates, rejects duplicates), get, packKeyOf, names.

Phase 4

Branch	File(s)	Lines	With	Into
`feature/p4-b-banking-pack`	`src/industries/banking/BankingPack.cpp`	60	G	`feature/p4-integration-banking`
`feature/p4-b-command-factory`	`src/engine/commands/CommandFactory.cpp`	70	-	`main`
`feature/p4-b-telecom-pack`	`src/industries/telecom/TelecomPack.cpp`	60	D	`feature/p4-integration-telecom`

- `BankingPack.cpp` : BankingPack factory methods.
- `CommandFactory.cpp` : Ctor registers the 7 verbs (lambdas), registerVerb, create (NotFoundError), verbs.
- `TelecomPack.cpp` : TelecomPack factory methods.

Phase 5

Branch	File(s)	Lines	With	Into
`test/p5-b-commands`	`tests/test_engine_commands.cpp`	50	-	`main`

Phase 6

Branch	File(s)	Lines	With	Into
`test/p6-b-facade`	`tests/test_engine_facade.cpp`	50	-	`main`

Phase 7: nothing new. Help review PRs and write docs, etc.

C. Connor

Phase 1

Branch	File(s)	Lines	With	Into
`feature/p1-c-progress-visitor`	`src/engine/items/WorkItemVisitor.cpp`	30	-	`main`
`feature/p1-c-work-context`	`src/engine/items/WorkContext.cpp`	35	-	`main`

- `WorkItemVisitor.cpp` : Visitor dtor and ProgressVisitor (virtual calls only, no link dependency on TaskItem).
- `WorkContext.cpp` : Constructor, accessors, markIrreversible()/irreversibleCount().

Phase 2

Branch	File(s)	Lines	With	Into
`feature/p2-c-flow-routing`	`src/engine/runtime/FlowControllerRouting.cpp`	95	D	`feature/p2-integration-runtime`
`test/p2-c-strategy`	`tests/test_pattern_strategy.cpp`	60	-	`main`

- `FlowControllerRouting.cpp` : NEW FILE. applyDecision, releaseEligible (ancestor walk), evaluateInstance.

Phase 3

Branch	File(s)	Lines	With	Into
`feature/p3-c-concrete-commands`	`src/engine/commands/ConcreteCommands.cpp`	100	F	`feature/p3-integration-work-commands`
`feature/p3-c-macro-command`	`src/engine/commands/MacroCommand.cpp`	70	B	`main`

- `ConcreteCommands.cpp` : NEW FILE. Start, Submit, Complete, Approve, Reject, Escalate, Cancel commands.
- `MacroCommand.cpp` : MacroCommand: all-or-nothing execute with reverse rollback, undo, isReversible, describe.

Phase 4

Branch	File(s)	Lines	With	Into
`test/p4-c-composite`	`tests/test_pattern_composite.cpp`	80	-	`main`
`test/p4-c-prototype`	`tests/test_pattern_prototype.cpp`	90	-	`main`

Phase 5

Branch	File(s)	Lines	With	Into
`feature/p5-c-tividy-core`	`src/engine/facade/Tividy.cpp`	90	A	`feature/p5-integration-tividy`

- `Tividy.cpp` : Ctor (ManualClock), dtor, installPack, packKeys/profile/packOf/workflowNames, startWorkflow, instanceIds/instance.

Phase 6

Branch	File(s)	Lines	With	Into
`test/p6-c-telecom-pack-smoke`	`tests/test_industry_telecom.cpp`	60	-	`main`

Phase 7: nothing new. Help review PRs and write docs.

D. Ruan

Phase 1

Branch	File(s)	Lines	With	Into
`test/p1-d-observer`	`tests/test_pattern_observer.cpp`	60	-	`main`

Phase 2

Branch	File(s)	Lines	With	Into
`feature/p2-d-flow-controller`	`src/engine/runtime/FlowController.cpp`	125	C	`feature/p2-integration-runtime`
`feature/p2-d-workflow-instance`	`src/engine/runtime/WorkflowInstance.cpp`	110	E	`feature/p2-integration-runtime`

- `FlowController.cpp` : Ctor, onEvent + FIFO drain (no recursion), react() reaction table, assignIfPossible, runAutomatic.
- `WorkflowInstance.cpp` : Ctor (member order matters), dtor, getters, findItem, itemsFor, addObserver, start, cancel, finish.

Phase 3

Branch	File(s)	Lines	With	Into
`test/p3-d-visitor`	`tests/test_pattern_visitor.cpp`	60	-	`main`
`test/p3-d-runtime`	`tests/test_engine_runtime.cpp`	50	-	`main`

Phase 4

Branch	File(s)	Lines	With	Into
`feature/p4-d-telecom-director`	`src/industries/telecom/TelecomDirector.cpp`	110	B	`feature/p4-integration-telecom`

- `TelecomDirector.cpp` : NEW FILE. ServiceActivationDirector::construct (items, routing, skip lists).

Phase 5

Branch	File(s)	Lines	With	Into
`test/p5-d-command`	`tests/test_pattern_command.cpp`	90	-	`main`

Phase 6

Branch	File(s)	Lines	With	Into
`feature/p6-d-tui-screens`	`src/ui/TuiScreens.cpp`	140	F	`feature/p6-integration-tui`

- `TuiScreens.cpp` : NEW FILE. macroBuilderScreen, historyScreen, progressScreen, undoScreen, advanceTimeScreen.

Phase 7: nothing new. Help review PRs and write docs.

E. Keval

Phase 1

Branch	File(s)	Lines	With	Into
`feature/p1-e-definition-validation`	`src/engine/definition/WorkflowDefinitionValidation.cpp`	95	F	`mai`

- `WorkflowDefinitionValidation.cpp` : NEW FILE. WorkflowDefinition::validationProblems(): the 8 rules in the header, one readable message each.

Phase 2

Branch	File(s)	Lines	With	Into
`feature/p2-e-definition-core`	`src/engine/definition/WorkflowDefinition.cpp`	35	F	`main`
`feature/p2-e-instance-snapshot`	`src/engine/runtime/WorkflowInstanceSnapshot.cpp`	60	D	`feature/p2-integratio runtime`
`feature/p2-e-state-defaults`	`src/engine/items/WorkItemState.cpp`	40	B	`feature/p2-integratio state-machine`
`feature/p2-e-task-work`	`src/engine/items/TaskItemWork.cpp`	50	F	`feature/p2-integratio state-machine`

- `WorkflowDefinition.cpp` : Ctor/dtor, getters, validate() (joins problems).
- `WorkflowInstanceSnapshot.cpp` : NEW FILE. InstanceMemento ctor/dtor, captureSnapshot, restoreSnapshot, sweepDeadlines().

- `WorkItemState.cpp` : WorkItemState dtor + the 11 default throwing operations (clear InvalidStateError text).
- `TaskItemWork.cpp` : NEW FILE. executeWork() template method, createMemento/restore/isReversible, default hooks.

Phase 3

Branch	File(s)	Lines	With	Into
`feature/p3-e-industry-pack`	`src/engine/facade/IndustryPack.cpp`	40	-	`main`
`feature/p3-e-workflow-builder`	`src/engine/definition/WorkflowBuilder.cpp`	100	A	`main`

- `IndustryPack.cpp` : IndustryPack dtor and every default factory method.
- `WorkflowBuilder.cpp` : WorkflowBuilder dtor + StandardWorkflowBuilder (stage stack, defaults, getResult validates and empties).

Phase 4

Branch	File(s)	Lines	With	Into
`test/p4-e-memento`	`tests/test_pattern_memento.cpp`	90	-	`main`
`test/p4-e-builder`	`tests/test_pattern_builder.cpp`	80	-	`main`

Phase 5: nothing new. Help review PRs and write docs.

Phase 6

Branch	File(s)	Lines	With	Into
`test/p6-e-abstract-factory`	`tests/test_pattern_abstract_factory.cpp`	90	-	`main`

Phase 7

Branch	File(s)	Lines	With	Into
`test/p7-e-ui`	`tests/test_ui.cpp`	60	-	`main`

F. Thabo

Phase 1

Branch	File(s)	Lines	With	Into
`feature/p1-f-iterator`	`src/engine/items/WorkItemIterator.cpp`	35	-	`main`

- `WorkItemIterator.cpp` : WorkItemIterator dtor; DepthFirstIterator (pre-order collected in the ctor through children(); first/next/isDone/currentItem, EngineError when done).

Phase 2

Branch	File(s)	Lines	With	Into
`feature/p2-f-task-item`	`src/engine/items/TaskItem.cpp`	90	E	`feature/p2-integration-state-machine`
`test/p2-f-adapter`	`tests/test_pattern_adapter.cpp`	70	-	`main`

- `TaskItem.cpp` : Ctor, prototype copy ctor, dtor, getters, find, bind/context, the 10 delegating lifecycle methods, accept, createIterator, counters, ManualTask.

Phase 3

Branch	File(s)	Lines	With	Into
`feature/p3-f-definition-instance`	`src/engine/definition/WorkflowDefinitionInstance.cpp`	25	E	`main`
`feature/p3-f-instance-command`	`src/engine/commands/InstanceCommand.cpp`	70	C	`featu integ work- comma`

- `WorkflowDefinitionInstance.cpp` : NEW FILE. createInstance(): validate, clone template, construct WorkflowInstance.
- `InstanceCommand.cpp` : NEW FILE. InstanceCommand execute (snapshot, try/catch, restore on failure), undo, isReversible, describe.

Phase 4

Branch	File(s)	Lines	With	Into
`feature/p4-f-demo-pack`	`src/industries/demo/DemoPack.cpp`	100	-	`main`
`test/p4-f-decorator`	`tests/test_pattern_decorator.cpp`	80	-	`main`

- `DemoPack.cpp` : DocumentApprovalDirector::construct and DemoPack factory methods.

Phase 5: nothing new. Help review PRs and write docs.

Phase 6

Branch	File(s)	Lines	With	Into
`feature/p6-f-tui-core`	`src/ui/Tui.cpp`	160	D	`feature/p6-integration-tui`
`test/p6-f-facade`	`tests/test_pattern_facade.cpp`	90	-	`main`

- `Tui.cpp` : Tui ctor, run, mainMenu, askChoice, profileFor, startWorkflowScreen, myWorkScreen.

Phase 7: nothing new. Help review PRs and write docs.

G. Jay

Phase 0 included Types.h, Outcome.h, WorkflowData.h, Clock.h, Participant.h, WorkflowEvent.h, Observer.h, ExternalConnector.h, WorkItemSpec.h, EscalationHandler.h, RoutingStrategy.h, AssignmentStrategy.h, Observers.h and general setup around the project, followed by management thereof.

Phase 1

Branch	File(s)	Lines	With	Into
`feature/p1-g-banking-adapters`	`src/industries/banking/BankingAdapters.cpp`	120	-	`main`
`feature/p1-g-client-profile`	`src/ui/ClientProfile.cpp`	15	-	`main`
`feature/p1-g-console`	`src/ui/Console.cpp`	55	-	`main`
`feature/p1-g-regional-assignment`	`src/industries/telecom/TelecomAssignment.cpp`	20	-	`main`
`feature/p1-g-telecom-credit-adapter`	`src/industries/telecom/TelecomCreditAdapter.cpp`	55	-	`main`
`feature/p1-g-telecom-network-adapter`	`src/industries/telecom/TelecomNetworkAdapter.cpp`	55	-	`main`

- `BankingAdapters.cpp` : CoreBankingVendor, CardSchemeVendor, CoreBankingAdapter, CardSchemeAdapter.
- `ClientProfile.cpp` : Default ctor (productName "TIVIDY") and term() with fallback to the engine word.

- `Console.cpp` : Console dtor + println; StdConsole; ScriptedConsole (echoes prompt+line into output()).
- `TelecomAssignment.cpp` : RegionalTechnicianAssignment::chooseParticipant.
- `TelecomCreditAdapter.cpp` : CreditBureauVendor (simulated, failNextCalls) + CreditBureauAdapter.
- `TelecomNetworkAdapter.cpp` : NetworkVendor + NetworkProvisioningAdapter (simulate_failures hook).

Phase 2: nothing new. Help review PRs and write docs.

Phase 3

Branch	File(s)	Lines	With	Into
`feature/p3-g-banking-tasks`	`src/industries/banking/BankingTasks.cpp`	130	-	`main`
`feature/p3-g-telecom-tasks`	`src/industries/telecom/TelecomTasks.cpp`	60	-	`main`

- `BankingTasks.cpp` : The 5 banking tasks.
- `TelecomTasks.cpp` : CreditCheckTask and ProvisionServiceTask (performWork, hooks, irreversible flag).

Phase 4

Branch	File(s)	Lines	With	Into
`feature/p4-g-banking-director`	`src/industries/banking/BankingDirector.cpp`	130	B	`feature/p4-integration-banking`

- `BankingDirector.cpp` : NEW FILE. DisputeDirector::construct (13 items, deadlines, routing, 9-item skip lists).

Phase 5: nothing new. Help review PRs and write docs.

Phase 6

Branch	File(s)	Lines	With	Into
`test/p6-g-banking-pack-smoke`	`tests/test_industry_banking.cpp`	60	-	`main`

Phase 7: nothing new. Help review PRs and write docs.

5. Test schedule (the "can I push yet?" table)

As mentioned, you can create your functional tests and store them locally, these are the checks for when you'll likely be able to verify your patterns are functional, and if so, push your tests.

ID	Test file	Owner	Opens in phase	Needs these merged first
T01	`tests/test_pattern_chain_of_responsibility.cpp`	B	1	nothing, push any time
T03	`tests/test_pattern_observer.cpp`	D	1	nothing, push any time
T02	`tests/test_pattern_strategy.cpp`	C	2	`regional-assignment`
T04	`tests/test_pattern_adapter.cpp`	F	2	`telecom-credit-adapter`, `telecom-network-adapter`, `banking-adapters`
T05	`tests/test_pattern_state.cpp`	A	3	`concrete-states`, `task-item`, `task-work`, `state-defaults`, `state-helpers`
T06	`tests/test_pattern_template_method.cpp`	B	3	`task-item`, `task-work`, `state-helpers`, `concrete-states`, `state-defaults`
T07	`tests/test_pattern_iterator.cpp`	A	3	`iterator`, `stage-item`, `task-item`
T08	`tests/test_pattern_visitor.cpp`	D	3	`progress-visitor`, `stage-item`, `task-item`
T17	`tests/test_engine_runtime.cpp`	D	3	`workflow-instance`, `instance-snapshot`, `flow-controller`, `flow-routing`, `deadline-monitor`,

ID	Test file	Owner	Opens in phase	Needs these merged first
				`action-history`, `task-item`, `concrete-states`
T09	`tests/test_pattern_composite.cpp`	C	4	`stage-item`, `task-item`, `macro-command`
T10	`tests/test_pattern_decorator.cpp`	F	4	`decorator-secure`, `decorator-validated`, `command-decorators`, `task-item`
T11	`tests/test_pattern_memento.cpp`	E	4	`instance-snapshot`, `instance-command`, `concrete-commands`, `task-work`
T12	`tests/test_pattern_prototype.cpp`	C	4	`definition-instance`, `workflow-builder`, `task-item`, `stage-item`
T13	`tests/test_pattern_builder.cpp`	E	4	`workflow-builder`, `definition-core`
T18	`tests/test_engine_items.cpp`	A	4	`work-context`, `item-memento`, `iterator`, `progress-visitor`, `work-item-base`, `task-item`, `task-work`, `state-defaults`, `state-helpers`, `concrete-states`, `stage-item`, `decorator-`

ID	Test file	Owner	Opens in phase	Needs these merged first
				`secure`, `decorator-validated`
T19	`tests/test_engine_definition.cpp`	A	4	`definition-validation`, `definition-core`, `definition-instance`, `workflow-builder`, `workflow-library`, `director-base`
T14	`tests/test_pattern_command.cpp`	D	5	`command-factory`, `action-history`, `concrete-commands`
T20	`tests/test_engine_commands.cpp`	B	5	`command-base`, `macro-command`, `command-decorators`, `instance-command`, `concrete-commands`, `command-factory`, `action-history`
T15	`tests/test_pattern_facade.cpp`	F	6	`tividy-core`, `tividy-actions`, `demo-pack`
T16	`tests/test_pattern_abstract_factory.cpp`	E	6	`tividy-core`, `tividy-actions`, `demo-pack`, `telecom-pack`, `banking-pack`
T21	`tests/test_engine_facade.cpp`	B	6	`industry-pack`, `tividy-core`, `tividy-actions`
T22	`tests/test_industry_banking.cpp`	G	6	`banking-director`, `banking-pack`,

ID	Test file	Owner	Opens in phase	Needs these merged first
				`tividy-core`, `tividy-actions`
T23	`tests/test_industry_telecom.cpp`	C	6	`telecom-director`, `telecom-pack`, `tividy-core`, `tividy-actions`
T24	`tests/test_industry_demo.cpp`	A	6	`demo-pack`, `tividy-core`, `tividy-actions`
T25	`tests/test_ui.cpp`	E	7	`tui-core`, `tui-screens`, `console`, `client-profile`

Folder tests (existing: connectors, core, events, policies; `tests/test_engine_items.cpp` already exists with the WorkItemSpec test, so A extends it) are minimal "is this folder broken?" checks (one happy path per class, one illegal use that must throw, etc.). Folder tests are spread across people, not given to one person: see the Owner column.

6. Integration branches (files that must merge together)

Branch	Lead	Phase	Who contributes	Merge when
`feature/p2-integration-runtime`	**D**	2	A (deadline-monitor, action-history), C (flow-routing), D (workflow-instance, flow-controller), E (instance-snapshot)	all helpers' branches merged in and the whole branch compiles together
`feature/p2-integration-state-machine`	**B**	2	B (state-helpers, concrete-states), E (task-work, state-defaults), F (task-item)	all helpers' branches merged in and the whole branch compiles together
`feature/p3-integration-work-commands`	**C**	3	C (concrete-commands), F (instance-command)	all helpers' branches merged in and the whole branch compiles together
`feature/p4-integration-telecom`	**D**	4	B (telecom-pack), D (telecom-director)	all helpers' branches merged in and the whole branch compiles together
`feature/p4-integration-banking`	**G**	4	B (banking-pack), G (banking-director)	all helpers' branches merged in and the

Branch	Lead	Phase	Who contributes	Merge when
				whole branch compiles together
`feature/p5-integration-tividy`	**C**	5	A (tividy-actions), C (tividy-core)	all helpers' branches merged in and the whole branch compiles together
`feature/p6-integration-tui`	**F**	6	D (tui-screens), F (tui-core)	all helpers' branches merged in and the whole branch compiles together

Helpers branch **from** the integration branch and open PRs **into** it. No CI runs on those PRs, so each helper runs `make` locally first. The full CI and the 4 approvals happen on the lead's PR into `main`. The lead creates the integration branch on day one of the phase. Make sure your branch is up to date with `main`, PR when you have a full picture ready to enter the project.

7. Dependency order in one picture

```
Phase 1  leaves (no dependencies)         -> WorkContext, Memento, Iterator, Command
base, adapters, Console... (G is busiest here)
Phase 2  core engine (needs phase 1)       -> runtime + state machine (integration),
StageItem, definition core
Phase 3  built on the engine               -> commands, decorators, builder, library,
industry tasks, IndustryPack
Phase 4  assembly                           -> industry directors + packs,
CommandFactory, DemoPack
Phase 5  facade                             -> Tividy
Phase 6  user interface                     -> Tui
Phase 7  last test                          -> ui folder test
```

8. Cut

If the schedule slips (we decide this around Phase 5-6), we can cut stuff out in this order (all patterns stay covered): Macro/Decorator/Retry commands, ValidatedWorkItem, Memento/undo, banking appeals, DataValueRouting, demo pack.